SnowPro Core COF-C03

Complete Study Guide

Domain	Topic	Weight
1	Snowflake AI Data Cloud Features & Architecture	25%
2	Account Access and Security	20%
3	Performance Concepts	15%
4	Data Loading and Unloading	10%
5	Data Transformations	20%
6	Data Protection and Data Sharing	10%

Verified against Snowflake documentation — September 2026

Domain 1: Snowflake AI Data Cloud Features & Architecture (25%)

1.1 Cloud Architecture (Three-Layer Model)

Storage Layer

- **Columnar storage** -- data stored in micro-partitions (50-500 MB compressed)
- Snowflake manages all storage; users cannot access underlying files directly
- Data is automatically compressed (no user action needed)
- Storage is billed separately from compute
- Data is stored in cloud provider's blob storage (S3, Azure Blob, GCS)

Compute Layer (Query Processing)

- **Virtual Warehouses** execute queries
- Warehouses are independent MPP clusters
- Warehouses can be started/stopped/resized without affecting other warehouses
- Scaling: **Scale Up** (bigger size) for complex queries, **Scale Out** (multi-cluster) for concurrency
- Warehouse sizes: X-Small, Small, Medium, Large, X-Large, 2XL, 3XL, 4XL, 5XL, 6XL

Cloud Services Layer

- Authentication and access control
- Infrastructure management
- Metadata management
- Query parsing and optimization
- **Billed only if usage exceeds 10% of total compute credit usage**

1.2 Snowflake Editions

Feature	Standard	Enterprise	Business Critical	VPS
Time Travel	1 day	90 days	90 days	90 days
Multi-cluster warehouses	No	Yes	Yes	Yes
Materialized views	No	Yes	Yes	Yes
Column-level security	No	Yes	Yes	Yes
Search optimization	No	Yes	Yes	Yes
Failover/Failback	No	No	Yes	Yes
Tri-Secret Secure	No	No	Yes	Yes
HIPAA/PCI DSS	No	No	Yes	Yes
Customer-managed keys	No	No	Yes	Yes
Dedicated VPS	No	No	No	Yes

1.3 Micro-Partitions

- Contiguous units of storage (50-500 MB compressed)
- Organized in a **columnar** format within each partition
- Snowflake automatically creates and maintains micro-partitions
- **Pruning**: Snowflake skips irrelevant partitions using min/max metadata
- You CANNOT manually control partitioning (Snowflake manages it)
- **Clustering keys** influence how data is organized within partitions

---

1.4 Data Types

Structured

- NUMBER, INT, FLOAT, DECIMAL
- VARCHAR, CHAR, STRING, TEXT
- BOOLEAN
- DATE, TIME, TIMESTAMP (TIMESTAMP_LTZ, TIMESTAMP_NTZ, TIMESTAMP_TZ)
- BINARY, VARBINARY

Semi-Structured

- **VARIANT** -- can store any JSON, Avro, ORC, Parquet, or XML value (up to 16 MB)
- **OBJECT** -- key-value pairs (like JSON object)
- **ARRAY** -- ordered list of values

1.5 Table Types

Type	Time Travel	Fail-Safe	Persists After Session	CDL Cost
Permanent	0-90 days (Enterprise+)	7 days	Yes	Full
Transient	0-1 day	0 days	Yes	Reduced
Temporary	0-1 day	0 days	No (session only)	Minimal
External	No	No	N/A (data external)	None

1.6 Views

Type	Definition Visible?	Uses Warehouse Credits?	Can Improve Performance?
Standard View	Yes (via SHOW/GET_DDL)	Yes (runs query each time)	No
Secure View	No (hidden from consumers)	Yes	No
Materialized View	Yes	Yes (for refresh)	Yes (precomputed)

1.7 Virtual Warehouses

Key Concepts

- Each size doubling approximately doubles credits/hour and compute power

- X-Small = 1 credit/hour, Small = 2, Medium = 4, Large = 8...
- **Auto-suspend**: shuts down after period of inactivity (default 10 min, minimum 1 min for non-serverless)
- **Auto-resume**: starts when a query is submitted
- Credit billing is per-second (60-second minimum)

Multi-Cluster Warehouses (Enterprise+)

- **Scaling Policy: Standard** (default) -- adds clusters aggressively, favors performance over cost (minimizes queuing)
- **Scaling Policy: Economy** -- adds clusters only when queuing is significant (~6 min estimated), favors cost over performance
- Min/Max cluster count controls

Warehouse Types

- **Standard** -- general-purpose
- **Snowpark-Optimized** -- for memory-intensive operations (ML, large data processing)

1.8 Resource Monitors

- Track credit usage by warehouse or account
- Actions at thresholds: **Notify**, **Notify & Suspend**, **Notify & Suspend Immediately**
- Can be set at account or warehouse level
- Only ACCOUNTADMIN can create resource monitors

1.9 Caching

Cache Type	Location	Duration	Reuse Condition
Metadata Cache	Cloud Services	Persistent	COUNT, MIN, MAX on same table
Result Cache	Cloud Services	24 hours	Same query, same role, no underlying data change
Local Disk Cache (Warehouse)	Compute Nodes	While WH running	Same warehouse runs query touching same data

Key exam point: Result cache does NOT consume warehouse credits. It is served from cloud services layer.

1.10 Stages

Stage Type	Scope	Location
User Stage (`@~`)	Per user	Internal (Snowflake-managed)
Table Stage (`@%table_name`)	Per table	Internal (Snowflake-managed)
Named Internal Stage (`@stage_name`)	Explicit object	Internal (Snowflake-managed)
Named External Stage (`@stage_name`)	Explicit object	S3, Azure Blob, GCS

1.11 Snowflake Features

Streams (Change Data Capture)

- Track DML changes (INSERT, UPDATE, DELETE) on a table
- Types: **Standard**, **Append-only**, **Insert-only** (for external tables)
- Creates a "change table" that records before/after row states

Tasks

- Schedule SQL execution (standalone or in DAGs)
- Can use cron or interval schedules
- Triggered by streams (WHEN SYSTEM$STREAM_HAS_DATA)
- Can use serverless compute (no warehouse needed)

Snowpipe

- Continuous, serverless data loading
- Triggered by cloud event notifications (S3 SQS, Azure Event Grid, GCS Pub/Sub)
- Uses serverless compute (no virtual warehouse)
- Near real-time loading (within minutes)

Stored Procedures

- Written in JavaScript, SQL, Python, Java, Scala
- Can run as **caller's rights** or **owner's rights**

UDFs (User-Defined Functions)

- Written in SQL, JavaScript, Python, Java, Scala
- Can be scalar or tabular (UDTFs)
- Cannot perform DML (unlike stored procedures)

Domain 2: Account Access and Security (20%)

2.1 Authentication Methods

Method	Description
Username/Password	Default, basic authentication
MFA	Multi-factor via Duo Mobile; can be enforced per user
SSO (SAML 2.0)	Federated via Okta, Azure AD, ADFS, etc.
Key-Pair Authentication	RSA 2048-bit; for service accounts and programmatic access
OAuth	External OAuth or Snowflake OAuth for clients

Key exam point: MFA is powered by Duo Security (built into Snowflake). Users must self-enroll or be enrolled by admin.

2.2 Role-Based Access Control (RBAC)

System-Defined Roles (Hierarchy)

```
ACCOUNTADMIN
├── SYSADMIN
│   └── (all custom roles should be granted to SYSADMIN)
├── SECURITYADMIN
│   └── USERADMIN
└── PUBLIC (granted to every user automatically)
```

Role Descriptions

Role	Purpose
ACCOUNTADMIN	Top-level role. Combines SYSADMIN + SECURITYADMIN. Use sparingly.
SYSADMIN	Create/manage databases, schemas, warehouses, and all objects
SECURITYADMIN	Manage grants, create/manage roles. Has MANAGE GRANTS privilege.
USERADMIN	Create/manage users and roles (but not grant privileges on objects)
PUBLIC	Automatically granted to all users. Minimal privileges.
ORGADMIN	Manage accounts within a Snowflake organization

Best Practices

- Custom roles should be granted to SYSADMIN (so SYSADMIN can manage their objects)
- ACCOUNTADMIN should NOT be default role for any user
- Use ACCOUNTADMIN only when necessary

- Grant least privilege needed

2.3 Privileges

Object Privilege Hierarchy

```
ACCOUNT → DATABASE → SCHEMA → Objects (TABLE, VIEW, STAGE, etc.)
```

Key Privileges

- **USAGE** -- required on database/schema before accessing any object inside
- **SELECT** -- read data from table/view
- **INSERT/UPDATE/DELETE** -- modify data
- **CREATE <object>** -- create new objects in schema
- **OWNERSHIP** -- full control, can grant privileges to others
- **OPERATE** -- start/stop/resume warehouses
- **MONITOR** -- view usage/history on warehouses

Future Grants

- Pre-grant privileges on objects that will be created in the future
- `GRANT SELECT ON FUTURE TABLES IN SCHEMA mydb.myschema TO ROLE analyst;`

2.4 Network Policies

- Restrict access by IP address (allow/deny lists)
- Can be applied at **account level** or **user level**
- User-level policies override account-level policies
- Use CIDR notation (e.g., 192.168.1.0/24)
- Only SECURITYADMIN or ACCOUNTADMIN can create network policies

---

2.5 Encryption

- **Always-on encryption** -- all data encrypted at rest and in transit
- AES-256 encryption for data at rest
- TLS 1.2 for data in transit
- **Automatic key rotation** every 30 days (Snowflake-managed keys)
- **Tri-Secret Secure** (Business Critical+): customer-managed key + Snowflake key = composite key
- **Periodic re-keying** (Enterprise+): re-encrypts data with new keys annually

---

2.6 Column-Level Security

Dynamic Data Masking (Enterprise+)

- Apply masking policies to columns
- Different users see masked/unmasked data based on role

- Policy is a SQL function that returns masked or real value
- Can be applied to multiple columns across tables

External Tokenization (Enterprise+)

- Replace sensitive data with tokens using external tokenization service

---

2.7 Row Access Policies (Enterprise+)

- Control which rows users can see
- Policy returns a boolean filter condition
- Applied to a table; evaluated at query time
- Can use mapping tables for complex access rules

2.8 Object Tagging

- Apply tags to objects for governance and classification
- Tag-based masking: attach masking policies to tags
- When a tagged column is queried, the masking policy fires automatically

---

2.9 Access History

- `SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY` view
- Records who accessed what data and when
- Includes direct and indirect access (through views, etc.)
- Retained for 365 days

Domain 3: Performance Concepts (15%)

3.1 Query Performance Optimization

Query Profile

- Visual tool in Snowsight showing query execution plan
- Shows operators, data flow, and performance statistics
- Key metrics: **bytes scanned**, **partitions scanned/total**, **bytes spilled**
- **Spilling** to local/remote storage = warehouse may be too small

Query History

- Available in Snowsight and via QUERY_HISTORY views
- Filter by user, warehouse, status, duration
- Shows execution time breakdown

3.2 Warehouse Sizing Strategy

Symptom	Action
Queries are slow (single large query)	**Scale Up** -- use larger warehouse
Too many concurrent queries queuing	**Scale Out** -- enable multi-cluster
Simple queries on small data are slow	Check caching; might be warehouse startup time
Queries spilling to disk	**Scale Up** -- more memory/compute needed

Key Sizing Facts

- Each size doubling = ~2x the compute power
- A query that takes 1 hour on X-Small might take ~30 min on Small
- Credit cost also doubles per size increase
- Bigger is NOT always better -- match warehouse to workload

3.3 Caching (Deep Dive)

Metadata Cache

- Maintained in cloud services layer
- FREE (no warehouse needed)
- Answers: COUNT(*), MIN(col), MAX(col), COUNT(DISTINCT col) on unchanged tables
- Works because Snowflake stores metadata about every micro-partition

Result Cache

- Exact same query + same role + data unchanged = cached result
- Valid for 24 hours
- FREE (no warehouse credits consumed)

- Resets if underlying data changes
- **Cannot be disabled** (always on)
- Works across warehouses (it's in cloud services)

Warehouse Local Disk Cache

- SSD cache on compute nodes
- Caches raw data read from storage
- Only available while warehouse is running (lost on suspend)
- Same warehouse must be used

3.4 Clustering

Micro-Partition Pruning

- Snowflake stores MIN/MAX values per column per micro-partition
- Filter predicates enable skipping irrelevant partitions
- Natural clustering: data loaded in order often clusters well on load-time columns

Clustering Keys (Enterprise+)

- Define columns to organize data within micro-partitions
- **Automatic Clustering** maintains the clustering over time (serverless, background)
- Best for: large tables (multi-TB) with known, frequent filter columns
- Costs credits for ongoing maintenance
- `SYSTEM$CLUSTERING_INFORMATION` -- check clustering quality
- `SYSTEM$CLUSTERING_DEPTH` -- check depth (lower is better, 1-2 is ideal)

When to Use Clustering Keys

- Table is very large (hundreds of GB+)
- Queries frequently filter on specific columns
- Clustering depth is high (data is poorly clustered)
- **Do NOT cluster** small tables or tables that are frequently fully scanned

---

3.5 Search Optimization Service (Enterprise+)

- Server-side feature for point-lookup queries (equality, IN, LIKE)
- Works on columns with high cardinality
- Serverless (Snowflake manages compute)
- Costs storage + compute credits
- Best for: selective lookups on columns NOT well-suited for clustering

---

3.6 Materialized Views (Enterprise+)

- Precomputed query results stored as a table
- Automatically refreshed when base table changes (serverless background)
- Best for: expensive queries on frequently-read, slowly-changing data
- Limitations:

- Based on a single table only (can be a regular table OR an external table)
- Limited SQL support (no joins, UDFs, HAVING, ORDER BY, LIMIT)
- Cannot be based on other views
- External table MVs require manual or auto-refresh of file metadata
- Cost: storage + serverless maintenance credits

3.7 Query Acceleration Service (Enterprise+)

- Offloads portions of a query to shared compute resources
- **Requires Enterprise edition or higher**
- Best for: queries with large scans that return small results
- Controlled by `ENABLE_QUERY_ACCELERATION` warehouse parameter
- Scale factor controls maximum resources (0-10, default 8; 0 = no limit)

3.8 Performance Best Practices Summary

- **1.** Right-size warehouses to workloads (dedicated WH per workload type)
- **2.** Enable auto-suspend and auto-resume
- **3.** Use clustering keys on large, frequently filtered tables
- **4.** Leverage result cache (ensure consistent query text)
- **5.** Use materialized views for expensive repeated computations
- **6.** Monitor query profile for spilling and full table scans
- **7.** Separate ETL and reporting warehouses

Domain 4: Data Loading and Unloading (10%)

4.1 Bulk Loading with COPY INTO

Basic Syntax

```
COPY INTO my_table

FROM @my_stage/path/

FILE_FORMAT = (TYPE = CSV FIELD_DELIMITER = ',' SKIP_HEADER = 1)

ON_ERROR = 'CONTINUE';
```

Key Options

Option	Values	Default
ON_ERROR	CONTINUE, SKIP_FILE, SKIP_FILE_n, ABORT_STATEMENT	ABORT_STATEMENT
PURGE	TRUE/FALSE -- delete files after successful load	FALSE
FORCE	TRUE/FALSE -- reload already-loaded files	FALSE
VALIDATION_MODE	RETURN_n_ROWS, RETURN_ERRORS, RETURN_ALL_ERRORS	(none)
SIZE_LIMIT	Max bytes to load (stops after threshold)	(none)
MATCH_BY_COLUMN_NAME	CASE_SENSITIVE, CASE_INSENSITIVE, NONE	NONE

Load Metadata

- Snowflake tracks which files have been loaded (64 days by default)
- Re-running COPY won't reload the same files (unless FORCE=TRUE)
- Use `LOAD_HISTORY` view to see load status

4.2 File Formats

Format	Structured?	Notes
CSV	Yes	Most common; configurable delimiters
JSON	Semi-structured	Loaded into VARIANT column or flatten during load
Parquet	Semi-structured	Columnar; efficient for analytics
Avro	Semi-structured	Row-based; schema embedded
ORC	Semi-structured	Columnar; common in Hadoop
XML	Semi-structured	Hierarchical

Named File Format Objects

```
CREATE FILE FORMAT my_csv_format
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  NULL_IF = ('NULL', 'null', '');
```

4.3 Staging Data

PUT Command (Internal Stages Only)

```
PUT file://C:/data/myfile.csv @my_stage AUTO_COMPRESS=TRUE;
```

- Uploads local files to internal stages
- Auto-compresses with gzip by default
- Only from SnowSQL or drivers (NOT Snowsight)

GET Command (Internal Stages Only)

```
GET @my_stage/myfile.csv.gz file://C:/downloads/;
```

- Downloads files from internal stage to local filesystem

4.4 Snowpipe (Continuous Loading)

How It Works

- **1.** Files land in a stage (S3, Azure Blob, GCS)
- **2.** Cloud event notification triggers Snowpipe
- **3.** Snowpipe loads data using serverless compute
- **4.** Near real-time latency (typically within minutes)

Key Facts

- **Serverless** -- no virtual warehouse required
- Uses a PIPE object with a COPY INTO definition
- Cost: per-file compute charge (cheaper than warehouse for small frequent loads)
- Cannot use VALIDATION_MODE
- Files are loaded in **best-effort order** (not guaranteed order)
- Use `SYSTEM$PIPE_STATUS` to check pipe health

REST API Alternative

- Call Snowpipe REST API `insertFiles` endpoint to trigger loading

- No cloud notifications needed (polling-based)

4.5 Data Transformation During Load

COPY INTO with SELECT

```
COPY INTO my_table (col1, col2, col3)
FROM (
  SELECT $1, $2, TO_DATE($3, 'YYYY-MM-DD')
  FROM @my_stage/data.csv
)
FILE_FORMAT = (TYPE = CSV);
```

- Can select specific columns, reorder, cast, transform
- Column references: `$1, $2, $3 ...` for CSV (positional)
- For semi-structured: `$1:field_name` for JSON/Parquet

4.6 Data Unloading

COPY INTO <location>

```
COPY INTO @my_stage/export/
FROM my_table
FILE_FORMAT = (TYPE = CSV COMPRESSION = GZIP)
HEADER = TRUE
OVERWRITE = TRUE
MAX_FILE_SIZE = 100000000;
```

Key Unloading Options

- **SINGLE** = TRUE/FALSE -- output as single file or multiple
- **MAX_FILE_SIZE** -- max size per output file (default 16 MB)
- **HEADER** -- include column headers (CSV)
- **OVERWRITE** -- replace existing files
- **PARTITION BY** -- partition output files by expression

4.7 Data Loading Best Practices

- **1. File sizing**: Aim for 100-250 MB compressed files

- **2. Number of files**: Match to number of warehouse nodes for parallel loading
- **3.** Use **dedicated loading warehouse** separate from query warehouse
- **4.** Enable **PURGE=TRUE** for automated cleanup in production
- **5.** Use **Snowpipe** for continuous, real-time-ish ingestion
- **6.** Use **VALIDATION_MODE** to preview errors before full load
- **7.** Prefer semi-structured native types (VARIANT) for JSON/Parquet

Domain 5: Data Transformations (20%)

5.1 DML Operations

INSERT

```
INSERT INTO target_table (col1, col2) VALUES (1, 'abc');

INSERT INTO target_table SELECT * FROM source_table;
```

UPDATE

```
UPDATE my_table SET status = 'active' WHERE created_date > '2024-01-01';
```

DELETE

```
DELETE FROM my_table WHERE status = 'inactive';
```

MERGE (Upsert)

```
MERGE INTO target t

USING source s ON t.id = s.id

WHEN MATCHED THEN UPDATE SET t.name = s.name

WHEN NOT MATCHED THEN INSERT (id, name) VALUES (s.id, s.name);
```

TRUNCATE

```
TRUNCATE TABLE my_table;  -- Removes all rows, resets table
```

Key difference: DELETE is DML (logged, recoverable with Time Travel). TRUNCATE is DDL (faster, also supports Time Travel).

5.2 Common Table Expressions (CTEs)

```
WITH sales_cte AS (

  SELECT region, SUM(amount) AS total_sales
```

```
  FROM sales
  GROUP BY region
)
SELECT * FROM sales_cte WHERE total_sales > 100000;
```

- Temporary, named result sets within a single query
- Improve readability of complex queries
- Can be referenced multiple times in the same query

5.3 Window Functions

Syntax

```
function_name() OVER (
  [PARTITION BY col1, col2]
  [ORDER BY col3 [ASC|DESC]]
  [ROWS|RANGE BETWEEN ... AND ...]
)
```

Common Functions

Function	Purpose
`ROW_NUMBER()`	Unique sequential number per partition
`RANK()`	Rank with gaps on ties
`DENSE_RANK()`	Rank without gaps on ties
`NTILE(n)`	Divide into n buckets
`LEAD(col, n)`	Value from n rows ahead
`LAG(col, n)`	Value from n rows behind
`FIRST_VALUE(col)`	First value in window
`LAST_VALUE(col)`	Last value in window
`SUM/AVG/COUNT()`	Running aggregates

QUALIFY Clause (Snowflake-specific)

```
SELECT id, name, region,
       ROW_NUMBER() OVER (PARTITION BY region ORDER BY sales DESC) AS rn
FROM employees
```

```
QUALIFY rn = 1;  -- Keep only top seller per region
```

- Filters results of window functions
- Like HAVING for GROUP BY, but for window functions
- Eliminates the need for subqueries

5.4 Semi-Structured Data

VARIANT Data Type

```
CREATE TABLE events (
  id INT,
  data VARIANT
);

INSERT INTO events SELECT 1, PARSE_JSON('{"name":"John","age":30}');
```

Querying Semi-Structured Data

```
-- Dot notation
SELECT data:name::STRING, data:age::INT FROM events;

-- Bracket notation
SELECT data['name']::STRING FROM events;

-- Nested
SELECT data:address.city::STRING FROM events;
```

FLATTEN (Critical for Exam!)

```
-- Array flattening
SELECT f.value::STRING AS item
FROM orders,
LATERAL FLATTEN(input => order_items) f;
```

```
-- Nested flattening

SELECT f.value:name::STRING

FROM events,

LATERAL FLATTEN(input => data:employees) f;
```

FLATTEN Output Columns

Column	Description
SEQ	Sequence number
KEY	Key for maps, index for arrays
PATH	Path to the element
INDEX	Array index (null for objects)
VALUE	The flattened value
THIS	The element being flattened

Type Casting from VARIANT

```
data:field::STRING     -- Cast to string

data:field::NUMBER     -- Cast to number

data:field::BOOLEAN    -- Cast to boolean

data:field::TIMESTAMP  -- Cast to timestamp
```

Always cast VARIANT fields to specific types for proper comparison and aggregation.

5.5 String Functions

Function	Example
CONCAT / \	\ ` CONCAT('a', 'b') or `'a' \ \ 'b'`
LENGTH	LENGTH('hello') → 5
UPPER / LOWER	Case conversion
TRIM / LTRIM / RTRIM	Remove whitespace
SUBSTR	SUBSTR('hello', 1, 3) → 'hel'
REPLACE	REPLACE('hello', 'l', 'r')
SPLIT_PART	SPLIT_PART('a-b-c', '-', 2) → 'b'
REGEXP_LIKE	Pattern matching

5.6 Date/Time Functions

Function	Description
`CURRENT_DATE()`	Today's date
`CURRENT_TIMESTAMP()`	Current timestamp
`DATEADD(part, n, date)`	Add interval to date
`DATEDIFF(part, d1, d2)`	Difference between dates
`DATE_TRUNC(part, date)`	Truncate to precision
`TO_DATE(str, fmt)`	String to date
`TO_TIMESTAMP(str, fmt)`	String to timestamp
`YEAR/MONTH/DAY(date)`	Extract component

5.7 Conditional Expressions

```
-- CASE

SELECT CASE WHEN status = 'A' THEN 'Active' ELSE 'Inactive' END;


-- IFF (Snowflake shorthand for simple CASE)

SELECT IFF(status = 'A', 'Active', 'Inactive');


-- COALESCE

SELECT COALESCE(col1, col2, 'default');  -- First non-null


-- NVL / NVL2

SELECT NVL(col1, 'default');             -- If null, use default

SELECT NVL2(col1, 'has value', 'null');  -- If not null / if null


-- NULLIF

SELECT NULLIF(col1, col2);  -- Returns NULL if equal


-- DECODE

SELECT DECODE(status, 'A', 'Active', 'I', 'Inactive', 'Unknown');


-- ZEROIFNULL / NULLIFZERO

SELECT ZEROIFNULL(col1);  -- NULL → 0
```

```
SELECT NULLIFZERO(col1);  -- 0 → NULL
```

5.8 Sequences

```
CREATE SEQUENCE my_seq START = 1 INCREMENT = 1;


-- Use in INSERT

INSERT INTO my_table VALUES (my_seq.NEXTVAL, 'data');


-- Sequences are NOT gap-free (gaps are normal)
```

5.9 Sampling

```
-- Row-based sampling (approximate % of rows)

SELECT * FROM my_table SAMPLE (10);  -- ~10% of rows


-- Block-based sampling (faster for large tables)

SELECT * FROM my_table SAMPLE BLOCK (10);  -- ~10% of blocks


-- Fixed row count

SELECT * FROM my_table SAMPLE (100 ROWS);
```

5.10 PIVOT / UNPIVOT

```
-- PIVOT: rows to columns

SELECT * FROM sales

  PIVOT(SUM(amount) FOR quarter IN ('Q1','Q2','Q3','Q4'));


-- UNPIVOT: columns to rows

SELECT * FROM pivoted_sales
```

```
UNPIVOT(amount FOR quarter IN (Q1, Q2, Q3, Q4));
```

Domain 6: Data Protection and Data Sharing (10%)

6.1 Time Travel

Overview

- Access historical data at any point within the retention period
- Available for tables, schemas, and databases

Retention Periods

Object Type	Standard Edition	Enterprise+
Permanent table	0-1 day	0-90 days
Transient table	0-1 day	0-1 day
Temporary table	0-1 day	0-1 day

Time Travel Queries

```
-- By timestamp
SELECT * FROM my_table AT(TIMESTAMP => '2024-01-15 10:00:00'::TIMESTAMP);


-- By offset (seconds)
SELECT * FROM my_table AT(OFFSET => -60*5);  -- 5 minutes ago


-- By query ID
SELECT * FROM my_table BEFORE(STATEMENT => '<query_id>');
```

UNDROP

```
UNDROP TABLE my_table;
UNDROP SCHEMA my_schema;
UNDROP DATABASE my_database;
```

- Restores the most recently dropped object with that name
- Only works within the Time Travel retention period

6.2 Fail-Safe

Aspect	Detail
Duration	7 days (starts after Time Travel expires)
Access	Snowflake support ONLY (not user-accessible)
Object types	Permanent tables only
Cost	Storage cost for the data during Fail-Safe period
Configurable?	No -- always 7 days, cannot be changed

Transient and temporary tables have NO Fail-Safe period.

Total Data Protection Timeline (Permanent Table, Enterprise+)

```
[Current Data] → [Time Travel: up to 90 days] → [Fail-Safe: 7 days]

Max total: 97 days of recovery window
```

6.3 Data Sharing

Secure Data Sharing (Provider → Consumer)

- **Zero-copy** -- no data movement or copying
- Provider creates a **SHARE** object and grants access to databases/schemas/tables/views
- Consumer creates a database FROM the share
- Real-time -- consumer always sees current data
- **Cross-region/cross-cloud**: requires replication

Share Objects

```
-- Provider side

CREATE SHARE my_share;

GRANT USAGE ON DATABASE mydb TO SHARE my_share;

GRANT USAGE ON SCHEMA mydb.public TO SHARE my_share;

GRANT SELECT ON TABLE mydb.public.my_table TO SHARE my_share;

ALTER SHARE my_share ADD ACCOUNTS = consumer_account;


-- Consumer side

CREATE DATABASE shared_db FROM SHARE provider_account.my_share;
```

Secure Views for Sharing

- Use **secure views** to share controlled subsets of data
- Definition is hidden from consumers

- Prevents consumers from deducing underlying data through query profiling

What Can Be Shared?

- Tables (permanent, dynamic, external, Iceberg -- not transient/temporary)
- Views (regular views, secure views, secure materialized views, semantic views)
- UDFs (both secure and non-secure)
- Cortex Search services
- Models (USER_MODEL, CORTEX_FINETUNED, DOC_AI)

Note: Snowflake strongly recommends sharing **secure** views and secure UDFs to prevent exposing sensitive data or view definitions to consumers.

Regular views CAN be shared but their definition is visible to consumers.

What CANNOT Be Shared?

- Transient/temporary tables
- Stages, pipes, streams, tasks
- Stored procedures

6.4 Reader Accounts

- Created by the provider for consumers who do NOT have a Snowflake account
- Provider creates and manages the reader account
- **Provider pays for compute** used by the reader account
- Reader accounts have limited functionality (read-only access to shared data)
- Reader accounts cannot create shares themselves

```
CREATE MANAGED ACCOUNT reader_acct

  ADMIN_NAME = 'admin'

  ADMIN_PASSWORD = 'P@ssword!'

  TYPE = READER;
```

6.5 Data Replication

Database Replication

- Replicate databases across regions and cloud providers
- Uses **primary** and **secondary** database model
- Secondary databases are read-only replicas
- Refresh is manual or scheduled

Account Replication (Business Critical+)

- Replicate account-level objects (users, roles, warehouses, resource monitors)
- Enables **Failover** and **Failback** for disaster recovery
- **Failover**: promote secondary to primary

- **Failback**: return to original primary after incident resolved

---

6.6 Data Encryption

At Rest

- AES-256 bit encryption (always on)
- Snowflake manages keys by default
- **Periodic re-keying** (Enterprise+): automatic annual re-encryption
- **Tri-Secret Secure** (Business Critical+): composite key from Snowflake + customer

In Transit

- TLS 1.2 (always on)
- All client connections are encrypted

6.7 Data Classification

- Automatic or manual classification of columns
- Identifies sensitive data (PII, PHI, financial)
- Tags columns with system tags:
- `SNOWFLAKE.CORE.SEMANTIC_CATEGORY` -- what kind of data
- `SNOWFLAKE.CORE.PRIVACY_CATEGORY` -- sensitivity level
- Integrates with masking policies

6.8 Cloning (Zero-Copy Clone)

```
CREATE TABLE clone_table CLONE source_table;

CREATE SCHEMA clone_schema CLONE source_schema;

CREATE DATABASE clone_db CLONE source_db;
```

Key Facts

- **Zero-copy**: no data is physically copied initially
- Clone is independent -- changes to clone don't affect source (and vice versa)
- Only new/modified micro-partitions consume additional storage
- Cloning is instant regardless of data size
- **Database/schema clones**: child object grants are preserved automatically
- **Individual table clones**: use `COPY GRANTS` to preserve grants
- `CREATE TABLE clone_t CLONE source_t COPY GRANTS;`
- Cloning a database clones all child objects

Time Travel Cloning

```
CREATE TABLE restored CLONE my_table AT(TIMESTAMP => '2024-01-15 10:00:00');
```

- Create a clone from a historical point in time